

99 Reinforcement Learning Agent

Casey Holden (holden.ca), Jade Gjermundsen (gjermundsen.j), Jack Young (joh.young),
Silas Spencer (spencer.si)

December 4, 2024

Abstract

99 is a three-player trick-taking card game with two phases, bidding and playing. Each player’s objective is to take the exact number of tricks in the playing phase as they bid in the bidding phase. Although there exists projects using reinforcement learning (RL) to play common card games such as blackjack and bridge, some features of 99 add a layer of complexity that makes this a particularly challenging problem to tackle. Using a Deep Q-Network (DQN), we attempted to train an RL agent to devise an optimal strategy for this game.

1 Introduction

Our group chose the trick-taking card game, Ninety-Nine (99) for the application of reinforcement learning (RL) algorithms. Unlike more widely studied card games such as blackjack and bridge, 99 involves two distinct phases: bidding and playing. During the bidding phase, players predict the number of tricks they will take, while in the playing phase, they aim to match their predictions exactly. This dual-phase structure, coupled with the need for strategic bidding and dynamic adaptation during gameplay, adds layers of complexity that make 99 an intriguing problem for RL research.

Our project’s purpose is to develop a reinforcement learning agent that can play the 99-card game effectively. Researchers have already extensively explored RL applications in simpler or more well-documented games, such as blackjack, which benefits from its straightforward rules and limited state space, and bridge, which has a rich repository of strategies and existing knowledge. However, the trick-based game named “99” remains relatively unexplored. A multifaceted gameplay offers a challenging and more complicated environment for strategizing and applying RL techniques.

This project contributes to the growing body of knowledge on applying RL to complex, multi-phase card games. By addressing the unique challenges posed by 99, we aim to advance the understanding of how RL agents can be designed and trained to handle games with intricate rule sets and strategic depth. Our findings provide valuable insights for future research in this area, highlighting the potential and limitations of using DQN for such applications. In this paper, we detail our methodology for implementing a DQN agent tailored to the nuances of 99, describe the training process, and discuss the challenges encountered. Then we present the hyperparameters and design decisions that have influenced our project. Our results demonstrate the feasibility of using DQN modeling to tackle the complexities of trick-taking strategies such as the 99-card game and offer a foundation for further exploration and refinement of RL techniques in similarly challenging environments.

2 Methods

Originally we planned to use a DQN accompanied by a Monte Carlo Search Tree to train our agent. However, this method was quickly deemed improbable due to the complexity of the game and sheer number of state spaces. Instead, we opted for a simple DQN. The architecture of our model is a fully connected feed-forward neural network consisting of three hidden layers consisting of 256 neurons, each accompanied by a ReLU activation function. This is followed by an output layer that maps the features from the last hidden layer to a vector of size 52, representing the number of actions the player can take in one turn. The DQN is trained to minimize the difference between these outputs and the target value for each action, given by the Bellman Equation.

$$Q(s, a) = E_{(s, a \rightarrow s', r) \sim \mathcal{H}} \left(r_{s, a} + \gamma \max_{a'} Q(s', a') \right)$$

Figure 1: Figure 1: The Bellman Equation as applied to the DQN

Additionally, batch training with replay memory was used to stabilize training. Because of the sequential nature of the game, training through sampled replay memory helps reduce correlation between consecutive experiences - which can negatively impact training.. Since the game involves multiple players, self-play was employed to train the agents and ensure that the agent was being continuously challenged rather than playing against a static opponent or unreliable human. Rewards are calculated based upon following equation:

$$\text{Reward} = \text{score}[\text{player 1}] - \text{score}[\text{player 2}] - \text{score}[\text{player 3}]$$

Thus incentivizing the agent to not only score a higher hand themselves but also to ‘hurt’ other players and lower their scores.

3 Data and Experiments

This section outlines the environment of our program along with the experiments performed and hyperparameters used during training.

3.1 Environment

We used a custom environment in order to represent our game state. The game is represented as follows:

Players and Cards:

The game involves three players. Each card is represented as an integer ranging from 0 to 51. The suit is computed using integer division (floor of the index of each card over thirteen). The rank of each card is computed using the modulo operator (remainder of the index of each card over thirteen). The cards are distributed among the players (thirteen each) and the remaining cards are set to 0 for bidding.

Key State Variables:

The two phases of the game, bidding and playing, are represented by 1 and 0 respectively. The trump suit is randomly selected at the start of a round (integer 0 to 3). Each player’s hand is represented by a numpy array of length 52. The value of each index represents the player’s relationships with that particular card; 1 for cards in the player’s hand, 0 for bid cards, and -1 for other cards unavailable to the player.

3.2 Experiments and Hyperparameters

Experiments performed and the hyperparameters used in training the reinforcement learning agent to play Ninety-Nine are described in this section. The central goal lies in exploiting a Deep Q Network (DQN) to improve the decision-making in the bidding and playing parts of the game.

Hyperparameters:

DQN’s hyperparameters’ attention is highly sensitive, and we needed to pay attention in achieving the balance of exploration, exploitation, and computing efficiency. Below are the key hyperparameters and their rationale:

- **Learning Rate (η):** To attain a tradeoff between the speed of convergence and stability of the model, we set the learning rate to =0.01.
- **Discount Factor (γ):** For long-term rewards while still abiding by immediate rewards, the discount factor was =0.99. In this sense, a multi-round game like Ninety Nine is critical.
- **Batch Size (β):** We also chose a batch size of b=64 to train. With this size of memory, there is sufficient variance in the sampled transitions and efficient gradient computation is possible.
- **Replay Memory Size:** Up to 200,000 transitions are stored in the replay memory. This ensures we get the model to learn from a variety of game scenarios thus allowing for more effective generalization.
- **Exploration Decay (ϵ):** The exploration parameter was set to 1.0 and it was used an exponential decay schedule during optimization.
1.0 and decaying to 0.05 over 500,000 steps:
(replacing this with a latex image: $t=\maxet,=\text{stepsln}(\min/\max)$)
- **Neural Network Architecture:** The DQN consisted of: an input layer matching the state vector size (e.g., 58 features combining player hands, bids, tricks, and trump suit), three fully connected layers with 256 neurons each, using ReLU activation, and an output layer of 52 nodes, representing the Q-values for each action.

Experiments:

To evaluate and optimize the agent’s performance, we conducted the following experiments:

- **Baseline Training:** At first, the training of the model was done on 50,000 steps random exploration for filling the replay memory. This set a standard of performance against which comparison could be made.
- **Exploration vs. Exploitation:** The exploration decay schedule was compared with an original fixed exploration rates by which the effect of dynamic exploration was measured. By using exponential decay, faster convergence and higher cumulative rewards were obtained.
- **Reward Function Variants:** Multiple reward formulations were tested, such as:
Simple incentives associated with the targets or goals set for bids either achieved or not. Variable pay system based on the number where tricks obtained is subtracted from tricks required. The latter was discovered to increase the sharpness of learning curves and the general performance of the scaled rewards.

Mathematical Formulation of Training

The DQN optimizes the loss function:

$$L(\theta) = E(s, a, r, s') D[(r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta))^2]$$

Where:

- θ : Weights of the DQN.
- θ : Weights of the target network (periodically updated).
- s, a, r, s' : State, action, reward, and next state, respectively.
- D : Replay memory.

4 Results

4.1 Evaluation Metrics

The agent was evaluated on:

- **Cumulative Reward:** Mean reward per game after 100 episodes of evaluation.
- **Success Rate:** Percentage of games in which the agent reached its bid.
- **Game Lengths:** Thus, the average number of steps per game was used to assess efficiency.
- **Comparison with Previous Models:** The proposed DQN was thus compared with a rule based agent and a random policy for purposes of verifying its supremacy. The DQN performed slightly better in accumulating total rewards and achievements above the chance level.

4.2 Key Findings

There was missing direct terminology' for important parts of the Ninety-Nine game namely the bidding phase. Unlike the playing phase with actions that had immediate consequence, the bidding phase posed a problem of strategic action planning in the absence of feedback, that is, success can only be judged at the end of the game. First, the AI found it difficult to differentiate between good bids and wild guesses because of the lack of this explicit mapping between actions and rewards. To solve this, we iteratively refined and explored the form of the reward function, starting from binary rewards to hindsight rewards scaled by the trick bid-trick taken difference and found this to generate smoother gradients when learning.

Encoding game phases as binary flags, and adding features like 'remaining tricks needed' made it easier to put abstract concepts like that in context. It was also interesting how the lack of consequences to define meant that the AI had to invent its own definitions, learn implicit relationships between inputs and objectives, and thereby emerge with behaviours such as strategic bluffing during bidding. Furthermore, this learning was supported with hyperparameter tuning, specifically exploration decay and replay memory, that allows the model to flexibly adapt in the lack of direct guidance. Finally, the project demonstrated the necessity of good reward engineering and state representation in reinforcement learning when working with abstract or undefined phases of a game.

5 Conclusion

By developing and training a DQN agent, we have navigated the unique challenges posed by the 99-card game’s dual-phase structure and strategic depth to demonstrate the application of Reinforcement Learning models on complex, multi-phase card games and competitive environments. Our team’s approach consisted of initially designing a fully connected feed-forward neural network (FFN), creating a Gymnasium-based environment to simulate the game, and then implementing a reward structure to guide the agent’s learning process.

Throughout the designing and implementation processes, our team observed that the DQN agent is able to learn to play 99 with a reasonable degree of competence. However, the performance of the agent is highly dependent on the hyperparameters and design choices established during the training process. Our results highlight the importance of careful tuning and the potential for further optimization. One of the key takeaways that emerged from our work is that the complexity of the 99-card game requires sophisticated state representation and action selection mechanisms, which our DQN model managed to address to some extent. Secondly, integrating more advanced techniques, such as Monte Carlo Tree Search (MCTS), could further enhance the agent’s performance by providing better exploration strategies and reducing the reliance on extensive training data.

Looking ahead into the future of this project, one promising avenue is to refine the reward structure and incorporate additional features that capture the nuances of the game more effectively. Another potential enhancement is to experiment with different RL algorithms, such as Proximal Policy Optimization (PPO) or Advantage Actor-Critic (A2C), to compare their effectiveness against DQN in this context. Additionally, incorporating self-play and batch training techniques could accelerate the learning process and improve the agent’s adaptability to diverse gameplay scenarios.

Overall, our project contributes to the understanding of how RL techniques can be applied to complex card games like 99. The insights gained from this work provide a foundation for further exploration and refinement, paving the way for more sophisticated and robust RL agents capable of mastering intricate and strategically demanding environments.

Github Repo: <https://github.com/holdenc20/NinetyNine.git>

6 References

- [1] Farama-Foundation. “Gymnasium/Gymnasium/Env/Tabular/Blackjack.py at Main · Farama-Foundation/Gymnasium.” *GitHub*, github.com/Farama-Foundation/Gymnasium/blob/main/gymnasium/envs/tabular/blackjack.py.
- [2] Parlett, David. “Ninety-Nine.” Ninety-Nine: An Original Card Game, www.parlettgames.uk/oricards/ninety9.html.
- [3] “Reinforcement Learning (DQN) Tutorial¶.” *Reinforcement Learning (DQN) Tutorial - PyTorch Tutorials 2.5.0+cu124 Documentation*, pytorch.org/tutorials/intermediate/reinforcement_q_learning.html.
- [4] Sutton, Richard S., and Andrew G. Barto. Reinforcement Learning: An Introduction. 2nd ed., The MIT Press, 2020.